

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Analytical Problem-Solving Assessment

Duration: 1hr

Code of Conduct:

I Vignesh S / 192325002 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vignesh S

ANALYTICAL PROBLEM-SOLVING ASSESSMENT — ANSWER

Question Chosen: A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply Dynamic Programming to develop an optimal policy and explain how value iteration can improve navigation efficiency. (5 Marks)

1. Modelling the Problem as an MDP

The warehouse floor is an 8×8 grid. Some cells are shelves (obstacles) the robot cannot enter; the goal cell is the packing station. The shortest-path task is modelled as a Markov Decision Process (MDP):

States (S): every free grid cell (row, column); shelf cells are excluded and the goal is a terminal state.

Actions (A): move North, South, East or West. A move into a wall or shelf leaves the robot in place.

Transition (P): movement is deterministic — each action leads to the intended neighbouring cell (or stays put if blocked).

Reward (R): -1 for every move, so accumulating the least negative return is exactly the same as taking the fewest steps. The goal is terminal with value 0 .

Discount (γ): 1.0 — with a step cost and a terminal goal the return of a state equals the negative of its distance to the goal, i.e. $V^*(s) = -(\text{shortest number of moves})$.

2. Dynamic Programming — the Bellman Optimality Equation

Dynamic Programming solves the MDP exactly using the Bellman optimality equation, which states that the value of a state equals the value of its best action:

$$V^*(s) = \max_a [R(s, a) + \gamma \cdot \sum P(s' | s, a) \cdot V^*(s')]$$

Because the transitions here are deterministic, the sum collapses and the update simplifies to $V^*(s) = \max_a [-1 + \gamma \cdot V^*(s')]$, where s' is the cell reached by action a . The optimal policy is then simply the action that moves to the highest-valued neighbour.

3. Value Iteration Algorithm (Python)

Value Iteration repeatedly applies the Bellman optimality update to every state until the values stop changing (the change falls below a tolerance θ). The implementation below uses only NumPy.

```
# Dynamic Programming (Value Iteration) for a warehouse robot's shortest path.
# The warehouse is an 8x8 grid. Shaded cells are shelves (obstacles). The robot
# must reach the GOAL (packing station) in the fewest moves. We model this as an
# MDP with reward -1 per move and a terminal GOAL, and solve it with Value
# Iteration using the Bellman optimality equation:  $V(s) = \max_a [ R + \gamma V(s') ]$ .
import numpy as np
SIZE=8; GAMMA=1.0
SHELVES={(1,1),(1,2),(1,3),(3,3),(3,4),(3,5),(5,1),(5,2),(5,5),(5,6),(2,6),(6,4)}
GOAL=(7,7); START=(0,0)
MOVES={0:(-1,0),1:(1,0),2:(0,-1),3:(0,1)} # N,S,W,E
ARROW={0:"↑",1:"↓",2:"←",3:"→"}
```

```
def nxt(s,a):
    r,c=s[0]+MOVES[a][0], s[1]+MOVES[a][1]
    if 0<=r<SIZE and 0<=c<SIZE and (r,c) not in SHELVES: return (r,c)
    return s
```

```
def value_iteration(theta=1e-6):
```

```

V=np.zeros((SIZE,SIZE)); deltas=[]; snap1=None
it=0
while True:
    delta=0; newV=V.copy()
    for r in range(SIZE):
        for c in range(SIZE):
            if (r,c)==GOAL or (r,c) in SHELVES: continue
            q=[-1 + GAMMA*V[nxt((r,c),a)] for a in MOVES]
            newV[r,c]=max(q); delta=max(delta,abs(newV[r,c]-V[r,c]))
    V=newV; deltas.append(delta); it+=1
    if it==1: snap1=V.copy()
    if delta<theta: break
# greedy policy
pol={}
for r in range(SIZE):
    for c in range(SIZE):
        if (r,c)==GOAL or (r,c) in SHELVES: continue
        q=[-1+GAMMA*V[nxt((r,c),a)] for a in MOVES]
        pol[(r,c)]=int(np.argmax(q))
return V,pol,deltas,snap1

V,pol,deltas,snap1=value_iteration()
np.save("V.npy",V); import json; json.dump({str(k):v for k,v in pol.items()},open("pol.json","w"))
np.save("deltas.npy",np.array(deltas))
print(f"Value Iteration converged in {len(deltas)} sweeps (theta=1e-6).")
print(f"V*(START={START}) = {V[START]:.0f} -> shortest path = {int(-V[START])} moves to goal.")
print(f"V*(goal-neighbour (7,6)) = {V[7,6]:.0f}")
print("\nAfter sweep 1, V of cells one step from the goal:")
for s in [(7,6),(6,7)]:
    print(f" V1{ s } = {snap1[s]:.0f}  (= -1 + V0(goal) = -1 + 0)")
print("\nConverged V*(s) grid (blank = shelf, 0 = goal):")
for r in range(SIZE):
    row=""
    for c in range(SIZE):
        if (r,c) in SHELVES: row+="  ##"
        else: row+=f"{V[r,c]:5.0f}"
    print(row)

```

4. Worked Computation

Starting from $V = 0$ everywhere, the first sweep sets every cell that is one move from the goal to -1 (since $-1 + V(\text{goal}) = -1$). Each subsequent sweep pushes this frontier one ring outward, so after k sweeps all cells within k moves of the goal hold their exact optimal value. The program's output is shown below.

Value Iteration converged in 15 sweeps (theta=1e-6).
 $V^*(\text{START}=(0, 0)) = -14 \rightarrow$ shortest path = 14 moves to goal.
 $V^*(\text{goal-neighbour } (7,6)) = -1$

After sweep 1, V of cells one step from the goal:

$V_1(7, 6) = -1$ ($= -1 + V_0(\text{goal}) = -1 + 0$)
 $V_1(6, 7) = -1$ ($= -1 + V_0(\text{goal}) = -1 + 0$)

Converged $V^*(s)$ grid (blank = shelf, 0 = goal):

```

-14 -13 -12 -11 -10 -9 -8 -7
-13 ## ## ## -9 -8 -7 -6
-12 -11 -10 -11 -10 -9 ## -5
-11 -10 -9 ## ## ## -5 -4
-10 -9 -8 -7 -6 -5 -4 -3
-9 ## ## -6 -7 ## ## -2
-8 -7 -6 -5 ## -3 -2 -1
-7 -6 -5 -4 -3 -2 -1 0

```

5. Optimal Policy and Value Function

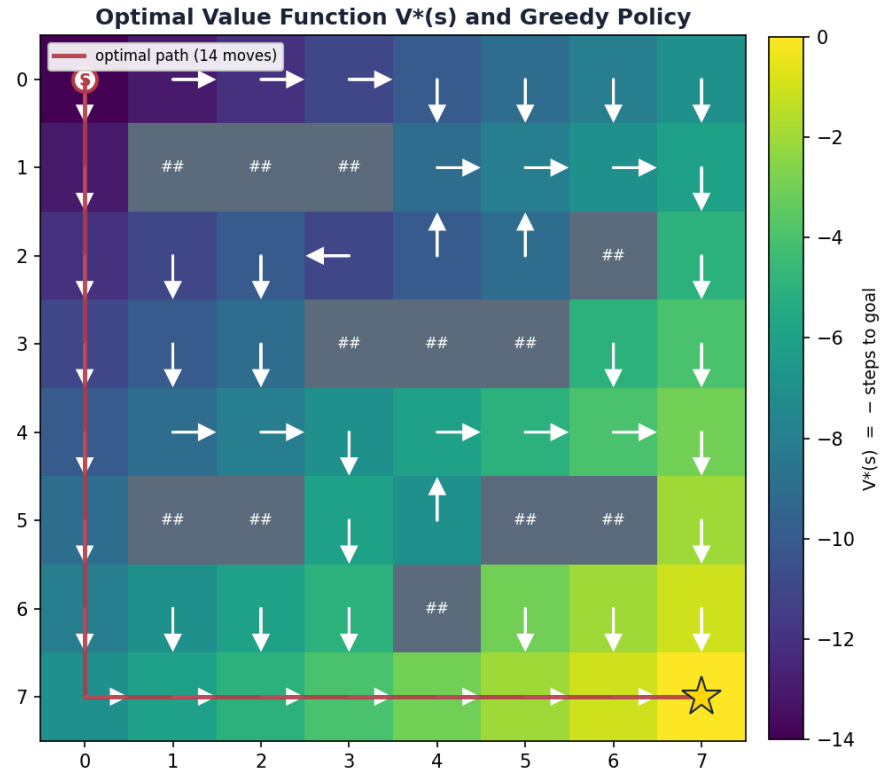


Figure 1: Converged optimal value function $V^*(s)$ (brighter = closer to goal). White arrows are the greedy optimal policy; the red line is the optimal 14-move path from Start (S) to the goal (★), routing around every shelf.

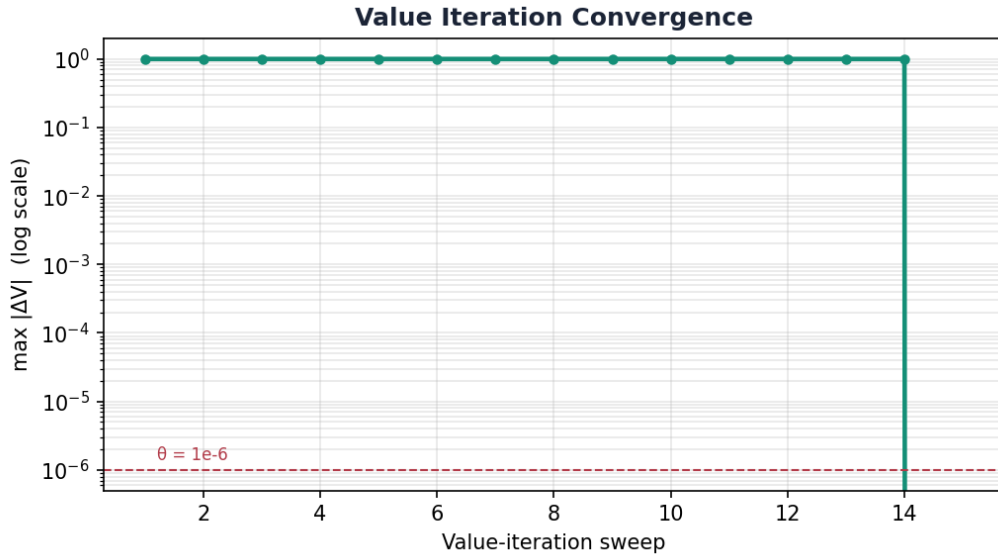


Figure 2: Convergence of Value Iteration — the maximum value change per sweep, which reaches the tolerance after 15 sweeps.

The value map decreases smoothly with distance from the goal, and the greedy policy derived from it always steps toward the neighbour with the highest value, producing the globally shortest obstacle-free route (14 moves from the start). Because $\gamma = 1$, each sweep resolves exactly one more ring of cells, so the maximum update stays at 1 until the frontier reaches the farthest reachable cell — after which nothing changes and the algorithm has converged.

6. How Value Iteration Improves Navigation Efficiency

Value Iteration yields a globally optimal policy rather than a locally greedy guess: every cell already stores the exact cost-to-go, so the robot never needs to search again at run time — it simply follows the arrows, guaranteeing the shortest obstacle-free path from any starting cell. A single solve therefore produces optimal routes for all possible start positions at once. If the layout changes (a shelf is added or the goal moves) the policy can be recomputed quickly, since Value Iteration costs only $O(\text{sweeps} \times |S| \times |A|)$ — here $15 \times 64 \times 4$ updates. Compared with a naïve greedy heuristic that can be trapped by obstacle configurations, the DP solution is provably optimal and complete.

7. Conclusion

By formulating warehouse navigation as an MDP and applying Dynamic Programming through Value Iteration, the robot obtains an optimal value function and the corresponding shortest-path policy that avoids all obstacles. The method is exact, converges in a small number of sweeps, and provides efficient, reusable navigation for the entire warehouse — demonstrating how value iteration improves navigation efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to	15	Responds	Responds	Responds	Unable to

Questions		accurately and confidently, extending answers with depth	correctly but with limited depth	partially; requires prompting	respond to questions effectively
-----------	--	--	----------------------------------	-------------------------------	----------------------------------